

מבני נתונים 2021 מועד א – סקיצה לפתרון + שגיאות נפוצות

שאלה 1, פתרון:

סעיף א:

נאתחל טבלת hash בגודל $O(n)$. שמגיע איבר נבדוק אם $(T^{\frac{1}{3}} - 3a_i)/2$ שלם ואם כן נחפש אותו בטבלת האש ואם נמצא נענה כן ואחרת לא. לאחר מכן תמיד נוסיף את a_i לטבלת ה hash. אפשר באופן אלטרנטיבי לבדוק בהגעת האיבר אם a_i בטבלה ואם כן לענות כן ואחרת לא. לאחר מכן להכניס ל hash את $(T^{\frac{1}{3}} - 2a_i)/3$ אם שלם. אפשרות נוספת היא בהגעת איבר לבדוק את $(T^{\frac{1}{3}} - 3a_i)$ ולענות כן אם נמצא ולאחר מכן להכניס את $2a_i$. אפשרות נוספת היא לבדוק אם $3a_i$ בטבלה ולאחר מכן להכניס את הערך $(T^{\frac{1}{3}} - 2a_i)$. כל הפעולות מבוצעות בתוחלת זמן קבוע לכל מפתח.

סעיף ב

נשתמש ב perfect hashing בגודל $O(n)$ שתכיל את כל n המפתחות בתוחלת $O(n)$ זמן. בהינתן שאילתה x נבדוק אם $(T^{\frac{1}{3}} - 3X)/2$ שלם ואם כן נחפש אותו בטבלת האש ואם נמצא נענה כן ואחרת לא. הבדיקה מתבצעת ב worst $O(1)$ case. באופן אלטרנטיבי אפשר להכניס רק את (עד n) המפתחות השלמים מבין $(T^{\frac{1}{3}} - 2a_i)/2$ בתוחלת זמן $O(n)$. בהינתן שאילתה x נבדוק אם ערכו נמצא בטבלה ולענות כן אם נמצא.

טעיות חוזרות:

סעיף א

לא התייחסו לכך שהביטוי עלול להיות לא שלם

קודם הכניסו את המפתח ורק אחרי זה בדקו אם הערך $(T^{\frac{1}{3}} - 3a_i)/2$ נמצא בטבלה (אז עלול להיות $j = i$)

לא תמיד הכניסו את a_i לטבלה

נוסחאות לא נכונות

בלבול של מה נכנס לטבלה ומה נבדק

סעיף ב

לא התייחסו לכך שהביטוי עלול להיות לא שלם

לא השתמשו ב perfect hash

השתמשו בפתרון שדורש מערך בגודל התחום שהוא עצום

עברו על המפתחות במקום לענות לשאילתה x

שאלה 2, פתרון:

סעיף א:

בסעיף זה נפעל על האיברים ומשקליהם כזוגות סדורים (x_i, w_i) כך שבכל צעד תמיד נדע לקשר ביניהם. לחלופין, ניתן לעבוד על מערך של איברים בודדים שמכילים מצביע לערך ולמשקל שלהם. כך-או-כך, בתיאור שלהלן נפשיט את הדיון ונתייחס לכך כמערך, שמכיל איברים עם משקל.

נגדיר קריאה רקורסיבית המקבלת מערך, ופרמטר של משקל מצטבר S . הקריאה הראשית תתבצע על מערך הקלט, עם $S = 0$. אופן הפעולה:

1. נמצא את חציון המערך לפי ערך (x_i) , נסמן את האינדקס שלו m (עבור median).
2. נבצע חלוקה (partition) של המערך סביב m כך שהאיברים משמאלו קטנים/שווים, ומימינו גדולים/שווים. הבהרה: לא אכפת לנו משוויונות, פרט לכך שאם לאחר הסידור החציון אינו במקום "שלו" כי יש יותר מדי ערכים x_i ששווים לו – נבחר את העותק שבמקום החציוני (גם ערכו x_m).
3. נסכם את המשקלים (w_i) של האיברים שמשמאל לחציון, נסמן סכום זה $T \equiv \sum_{i < m} w_i$, ונבדוק:
 - a. אם $T + S \geq \frac{1}{2}$: נעתיק את האיברים שמשמאל לחציון למערך חדש, ונפעיל עליו את הקריאה הרקורסיבית עם המשקל המצטבר S .
 - b. אחרת $(T + S < \frac{1}{2})$ אם $T + S + w_m \geq \frac{1}{2}$: נחזיר את x_m כערך המבוקש.
 - c. אחרת $(T + S + w_m < \frac{1}{2})$: נעתיק את האיברים שממין לחציון למערך חדש, ונפעיל עליו את הקריאה הרקורסיבית עם המשקל המצטבר $S + T + w_m$.

נשים לב שמספר האיברים מצטבר אחד לזה שאחריו קטן לפחות פי 2, כי סילקנו את החציון ואת אחד הצדדים.

נכונות האלגוריתם נובעת מכך שהחציון מפריד בין האיברים הגדולים והקטנים של המערך, וסיכום המשקלים בודק את התנאי הנחוץ, כלומר אם החציון הנוכחי הוא הראשון שהמשקל המצטבר עד אליו וכולל, חצי לפחות.

זמן הריצה בהתעלם מקריאות רקורסיביות הוא $\Theta(n)$, עבור: מציאת החציון באמצעות SELECT (אלג' החציונים), ביצוע חלוקת המערך מתבצע בזמן ליניארי, וגם סיכום המשקלים שבצעד-3 (זה תטא כי נגענו בכל איבר לפחות פעם אחת עבור החלוקה). לאחר מכן, תיתכן קריאה רקורסיבית. לפיכך נוסחת הרקורסיה היא:

$$T(n) \leq T\left(\frac{n}{2}\right) + \Theta(n) \Rightarrow T(n) = \Theta(n)$$

לפי משפט המאסטר, עץ רקורסיה, או "סתם" טור הנדסי מתכנס.

סעיף ב:

סעיף זה מעט פשוט יותר, כי אין ערכים ומשקלים – אלא ערכים בלבד. תבנית הפתרון זהה, פרט לתיקוני הבדיקה הנדרשים: 0 במקום $\frac{1}{2}$, והשוויון בתנאים "עובר צד". נפרט:

קריאה רקורסיבית ביחס למערך קלט ופרמטר S . הקריאה הראשית תתבצע על המערך המקורי עם $S = 0$:

1. נמצא את חציון המערך (a_i) נסמן את האינדקס שלו m (עבור median).
2. נבצע חלוקה (partition) של המערך סביב m כך שהאיברים משמאלו קטנים/שווים, ומימינו גדולים/שווים. הבהרה: לא אכפת לנו משוויונות, פרט לכך שאם לאחר הסידור החציון אינו במקום "שלו" כי יש יותר מדי ערכים x_i ששווים לו – נבחר את העותק שבמקום החציוני (גם ערכו x_m).
3. נסכם את ערכי האיברים (a_i) שמשמאל לחציון, נסמן סכום זה $T \equiv \sum_{i < m} a_i$, ונבדוק:
 - a. אם $T + S > 0$: נעתיק את האיברים שמשמאל לחציון למערך חדש, ונפעיל עליו את הקריאה הרקורסיבית עם המשקל המצטבר S .

- b. אחרת ($T + S \leq 0$) אם $T + S + a_m > 0$: נחזיר את a_m כערך המבוקש.
- c. אחרת ($T + S + a_m \leq 0$): נעתיק את האיברים שמימין לחציון למערך חדש, ונפעיל עליו את הקריאה הרקורסיבית עם הסכום המצטבר $S + T + a_m$.

נימוק הנכונות, וניתוח הסיבוכיות, זהים לסעיף א'.

וריאציות לפתרון:

- ניתן לעבוד על המערכים in-place באמצעות מצביעים לתחילת וסוף המקטע ה"פעיל" במקום להעתיק למערך חדש בכל צעד. (דורש עוד הסבר מילולי ושני פרמטרים נוספים).
- בסעיף א' ניתן לתחזק שני סכומים, אחד של המשקלים מימין ואחד של המשקלים משמאל, ולבדוק שהמשקל הימני קטן-שווה חצי במקום לבדוק שהמשקל השמאלי בתוספת משקל החציון חצי ומעלה.
- "טריק" בסעיף א': ניתן לוותר על פרמטר המשקלים המצטברים כך: כאשר נרצה להתקדם ברקורסיה לצד השמאלי של המערך, נחליף את המשקל של החציון במשקל הכולל שלו ושל הצד הימני, והרקורסיה תפעל על הצד השמאלי יחד עם החציון. באופן דומה אם נתקדם ימינה, נגדיל את משקל החציון במשקל הכולל של הצד השמאלי. חשוב לשים לב שבמקרה זה אין לנו תנאי עצירה "מידי", וגם לא מובטח שנסלק לפחות חצי מאיברי המערך. לכן צריך לטפל ידנית במקרי הבסיס של מערך בגודל 1 או גודל 2:
 - גודל 1: האיבר היחיד הוא האיבר המבוקש.
 - גודל 2: אם האיבר הראשון שוקל חצי או יותר – נחזיר אותו. אחרת, זה האיבר השני.
- בסעיף ב' ניתן ליישם את הטריק, אך בזהירות רבה יותר. הסיבה לכך היא ששינוי המשקולות בסעיף א' אינו פוגע בערך המבוקש, בעוד שבסעיף ב' עלינו להחזיר בסופו של דבר את הערך הנכון. לכן בסעיף ב' יש לנמק מדוע שינוי ערך החציון עקב הטריק אינו פוגע בסדר שלו במערך, או בערך החזרה הסופי (הסיבה היא שאם ביצענו את הטריק, סימן שהסכום עד החציון וכולל הוא אי-חיובי, לכן בוודאות לא נחזיר אותו בהמשך).

טעויות גדולות:

- מיונים
 - פתרון בהנחה שהמערך ממורן... הוא לא.
 - הנחה שניתן למיין אותו בזמן ליניארי radix/bucket sorts: לא ידוע דיוק המשקלים.
- בלבולי ערכים ומשקלים
 - התעלמות מערכי ה- x_i , ובמסגרתה חיפוש חציון לפי משקלים, כך שהאיבר ה"מבוקש" הוא זה שמשקלו מינימלי כך שסכום המשקלים עד למשקלו מגיע לחצי.
 - התעלמות ממשקולות, וסיכום ערכי איברים והשוואתם לחצי.
- הכפלת הערך במשקל, וטענה שהחציון של מערך המכפלות הוא החציון הממושקל.
- רדוקציה מסעיף ב' לסעיף א' על-ידי המצאת משקולות לכל איבר לפי ערכו חלקי הסכום, כדי לקבל סכום משקלים מנורמל ל-1. (זה לא יעבוד)
 - בפרט, חמורה יותר הרדוקציה הזו אם לא מנפים הצידה את הערכים השליליים לסכום נפרד, כי אז המשקולות בכלל אינן בטווח (0,1).

טעויות בינוניות וקטנות:

- ענייני נכונות ומקרי קצה
 - התעלמות מסכום-מצטבר (משקלים בסעיף א', ערכים בסעיף ב') בעת הקריאה לרקורסיה, או התחשבות שגויה בו, כגון לשכוח להתייחס גם לתרומת החציון כשמסלקים אותו או איפוסו בתנועה שמאלה.
 - המחשבה שיש להמשיך תמיד רק לצד ימין, או רק לצד שמאל (מקרה "ראשי" חסר).
 - חוסר טיפול בתנאי העצירה של הרקורסיה, או תנאי ברור שמעיד על החזרת ערך.

- "חורים" בתרחישים המטופלים. דוגמה נפוצה: בדיקה אם ערך נתון הוא חיובי או שלילי בלבד (יש גם אפס). (לא מדובר על "אם... אחרת..." אלא על "אם חיובי: ..., אם שלילי: ...").
- טעות בהחזרת הערך הנכון כאשר סכום המשקלים שלפני החציון הוא חצי בדיוק (אזי החציון הממושקל הוא הקודם-לחציון, ולא החציון), וסוגיות דומות סביב בדיקות שיוויון שגויות.
- היעדר אתחול/הגדרה של פרמטרים לקריאה הרקורסיבית, או ערכיהם בקריאה הראשית.
- ענייני סיבוכיות
 - שימוש לקוי באופן בוטה בחסמים תחתונים עליונים, כגון: $O(n) = \Omega(1)$.
 - בחירת החציון של המשקלים לצורך ביצוע החלוקה (אך עדיין ביצוע חלוקה לפי ערכי x_i): מניב סיבוכיות $O(n^2)$.
 - בחירת איבר ציר (pivot) אקראי לצורך ביצוע החלוקה.
 - ניתוח סיבוכיות חסר, או שגוי.

טעויות שלא איבדו ניקוד:

- בלבולי מתמטיקה בין טורים הנדסיים והרמוניים.
- אזכור לאלגוריתם quickselect לא איבד ניקוד, על-סמך ההנחה שהכוונה ל-SELECT או לתהליך ה-partition שמבצע בהינתן שיש לנו איבר ציר ביד (שימוש באלגוריתם בניתוח סיבוכיות איבד ניקוד).

שאלה 3, פתרון:

סעיף א:

נאתחל מערך פלט בגודל n .

נשים לב שהאיבר המינימלי יכול להימצא ב- $k + 1$ איברים הראשונים. נבנה ערימת מינימום מ- $k + 1$ האיברים הראשונים וכעת במשך n איטרציות נכניס את המינימום בערימה למיקום הבא במערך הפלט ונכניס את האיבר הבא במערך הקלט לערימה (אלא אם הגענו כבר לסוף מערך הקלט ואז לא נכניס).

גודל הערימה בכל זמן $O(k)$ ולכן כל פעולה תקח $O(\log k)$ וסהכ נקבל סיבוכיות זמן $O(n \log k)$.

סעיף ב:

נבצע חסם תחתון באמצעות עץ השוואות. נחסום כמה עלים יכולים צריכים להיות בכל עץ השוואות. ניקח מערך ממין ונחלקו לבלוקים בגודל $\sqrt{n}$ נשים לב שאם נסדר כל בלוק בכל פרמוטציה שנבחר המערך שנקבל יקיים את תנאי השאלה. ולכן מספר המערכים הוא לפחות $(\sqrt{n}!)^{\sqrt{n}}$. כל מערך כזה צריך להוביל לעלה שונה ולכן נקבל שיש $(\sqrt{n}!)^{\sqrt{n}}$ עלים בכל עץ השוואות. ולכן נדרש לפחות ל $\log((\sqrt{n}!)^{\sqrt{n}}) = \Omega(n \log n)$ השוואות.

טעויות חוזרות

סעיף א –

חלק חילקו את המערך למקטעים זרים בגודל $2k$ (או k או דברים דומים) ומיינו כל מקטע בנפרד וטענו שהמערך ממין לאחר מכן. שימו לב שהמערך לא בהכרח ממין (לדוגמא האיבר ה- $K+5$ בגודלו יכול להופיע במיקום ה- $2k+1$ במערך הלא ממין ואז מכיוון שימיון עם המקטע $2k+1..4k$ הוא יוכל לשבת רק במיקום $2k+1..4k$ וזה לא המקום הנכון עבורו).

סעיף ב-

לבצע רדוקציה למיון כאשר הרדוקציה עצמה לוקחת $O(n \log n)$ זמן או אפילו יותר

חלק קבלו מערך בגודל $\sqrt{n}$ והפעילו עליו את סעיף א – שימו לב שזה לא בדיוק עונה על השאלה. השאלה תראו שעבור מערך בגודל n (ולא $\sqrt{n}$) ועבור $k = \sqrt{n}$ לא ניתן למיין בזמן קטן אסמפטוטית מ $n \log n$.

חלק ניסו לחשב את כמות העלים בעץ השוואות והיו מגוון טעויות בחישוב כמו לא להתייחס שלכל איבר (למעט ה- k הראשונים והאחרונים) יש $2k$ אפשרויות למיקום אבל המיקומים תלויים אחד בשני.

שאלה 4, פתרון:

נשתמש בעץ AVL עם השוואה לפי גודל הזווית, כאשר נתאר תחילה את הפתרון ללא Rotate ולבסוף נסביר איך להוסיף תמיכה בפעולה זו. הכנסה, מחיקה וחיפוש למדנו בכיתה שניתן לבצע ב- $O(\log n)$. את פעולת מחיקת האיברים על קשת נממש בצורה דומה לפתרון שאלה 4 מתרגול 6. נניח תחילה כי הנקודה θ_1 קטנה מהנקודה θ_2 . אם הנקודות לא קיימות בעץ, נוסיף אותן. נבצע split לפי הנקודה θ_1 לקבלת T_1, T_2 כאשר T_1 מכיל נקודות קטנות יותר ו- T_2 גדולות יותר. לאחר מכן נבצע split על T_2 לפי הנקודה θ_2 לקבלת T'_1, T'_2 כאשר T'_1 מכיל נקודות בין θ_1 לבין θ_2 ו- T'_2 מכיל נקודות גדולות יותר מ- θ_2 . לסיום, נבצע פעולת join בין T_1 ו- T'_2 בעזרת המפתח θ_1 שבסופה נמחק אותו מהעץ. כפי שלמדנו בכיתה, עלות פעולות split, join היא $O(\log n)$ ולכן זוהי גם עלות הפתרון. במידה והנקודה θ_1 גדולה מהנקודה θ_2 , נחליף את הסדר ביניהן ונעשה אותו דבר כמו קודם רק שנחזיר את העץ המשלים T'_1 , כיוון שמדובר בקשת המשלימה. כדי לתמוך בפעולה Rotate יש לשמור משתנה עזר שיצבור את הזוויות שנקלטו בפעולה זו, מודולו 2π . את כל הזוויות שמקבלים מהמשתמש בשאר הפעולות נתקן ע"י חיסור של משתנה זה, מודולו 2π . לכן סיבוכיות פעולה זו היא $O(1)$ והיא לא משפיעה לרעה על סיבוכיות שאר הפעולות.

טעות מאוד נפוצה: לא לוקחים בחשבון את האפשרות של מחיקה מעגלית כך ש- $\theta_1 > \theta_2$. מאחר ובפועל אנחנו מחסרים מהן את ההיסט (מודולו 2π) לפני שעובדים איתן, ההנחה עוד יותר בעייתית.

טעות קטנה יותר הייתה להפעיל split פעמיים על העץ המקורי, למרות שהוא נהרס אחרי ההפעלה הראשונה.

אעיר שהרבה סטודנטים השתמשו בעץ AVL עם שדות size, אבל אין צורך.

שאלה 5, פתרון:

(בשאלה זאת "לא יודע" על סעיף קיבלה 2 נק, על 2 סעיפים 3 נק ועל 3 סעיפים 5 נק)

א. (4 נק על ההוכחה, 3 נק על ניתוח הסיבוכיות)
נוכח עבור המימוש `build_heap1`.
נבחין כי נבצע `heapify_down` על צומת אם רק אם ביקרנו בבנו השמאלי (כלומר אם האינדקס j שווה לאינדקס של הבן השמאלי). נוכח שנבקר בכל צומת בדיוק פעם אחת באינדוקציה על השכבות. עבור השכבה התחתונה (עלים) זה מידי בגלל מבנה הלולאה. צעד האינדוקציה נובע מכך שנבקר בצומת רק אחרי שביקרנו את בנו השמאלי ומהנחת האינדוקציה זה יקרה פעם אחת.
ההוכחה עבור `build_heap2` מחליפה בין המילים שמאל וימין וזוגי ואי זוגי.
ניתוח סיבוכיות – מבצעים אותן פעולות `heapify_down` שהפעולה `build_heap` עושה. ניתחנו בשיעור שזמן הריצה מסתכם ל $O(n)$.

ב. הפרכה. למשל הערימה הבינארית [7,6,5,4,3,2,1]. העקרון להפרכה הוא שנבצע `heapify_down` לשורש בשלב די מוקדם של האלגוריתם (ונבצע זאת רק פעם אחת) ולכן אם לא ביקרנו במינימום עד כה אז הוא לעולם לא יגיע לשורש.

ג. הוכחה. אנו עושים `heapify_down` לצומת (מיד לאחר שעשינו `heapify_down` לבנו הימני) רק לאחר שעשינו `heapify_down` לבנים שלו. כמו שראינו בשיעור, תכונה זאת מבטיחה שלאחר `heapify_down` בהכרח תת העץ של אותו הצומת יקיים את כלל הערימה.

שגיאות נפוצות:

סעיף א –

1. רבים שכחו לנתח את הסיבוכיות.
2. רבים הניחו ש `heapify_down` לוקח זמן קבוע.
3. פתרונות רבים ניסו לנתח את האלגוריתם באמצעות התייחסות לכלל הערימה והמפתחות. השיקולים האלו לא רלוונטיים לסעיף (שגויים) מכיוון שרק נשאל על כמות הקריאות לפעולה `heapify_down` שיש בפסאודו קוד והכמות הזאת בלתי תלויה בשינויים שמתחרשים בערימה.
4. ניסיונות שגויים להוכיח באינדוקציה. היו ניסיונות לומר שתתי העצים של השורש הן ערימות בינאריות שעליהן רץ האלגוריתם. הבעיה היא שמבחינת הזכרון הערימות האלו לא יושבות על מקטע רציף.

סעיף ב –

1. רבים ניסו להוכיח.
2. היו ניסיונות הפרכה שהאלגוריתם כן מצליח עליהם.

סעיף ג –

1. רבים ניסו להפריך.
2. רבים ניסו להוכיח רק על סמך כך שביצענו `heapify_down` על כל צומת (תנאי שסעיף ב מראה שלא מספיק).
3. ניסיונות אינדוקציה שגויים בדומה לסעיף א.